



Tecnológico Nacional de México Campus Hermosillo

Grupo:

“S6B”

Título

Sistema Hotelero

Alumnos

Emmanuel Yave Castro Espinoza
Martin Zahid Aburto Gonzalez
Marco David águilar moreno

Profesor

MOROYOQUI IBARRA JONATHAN ADRIAN

Materia

PROGRAMACIÓN WEB

Hermosillo, Sonora.

28 de Mayo del 2026

--- 6.2 INTRODUCCIÓN ---

Descripción general del sistema desarrollado

El sistema desarrollado consiste en un ecosistema web integral para la gestión hotelera, estructurado a través de una API en el Back-End (.NET) y una interfaz interactiva en el Front-End (React). La aplicación centraliza la administración de catálogos base indispensables —como clientes, servicios y habitaciones— y los integra con flujos transaccionales avanzados que abarcan desde el control de reservaciones en tiempo real hasta la simulación completa del cierre de cuentas y facturación de huéspedes.

Problema que resuelve o necesidad que cubre

Este sistema automatiza y resuelve las complicaciones operativas tradicionales de un complejo hotelero mediante soluciones tecnológicas específicas:

- **Colisiones en reservaciones:** Previene la sobreventa de habitaciones mediante validaciones estrictas en el servidor que aseguran que ningún cuarto sea agendado si ya se encuentra ocupado en el rango de fechas seleccionado.
- **Falta de control en consumos extras:** Elimina el registro disperso de cargos adicionales (como spa, restaurante o tours) al asociar directamente los consumos del catálogo a las reservaciones activas de los huéspedes.
- **Opacidad administrativa:** Sustituye los procesos manuales de facturación mediante pantallas de Check-Out automatizadas que calculan totales precisos con impuestos (IVA) , acompañadas de un Dashboard con indicadores de ocupación en tiempo real.

Objetivo general del proyecto

Construir una plataforma digital modular y escalable que optimice la gestión de operaciones hoteleras diarias, garantizando la consistencia de los datos mediante una arquitectura robusta en el servidor , facilitando una experiencia de usuario fluida e interactiva en la interfaz visual , y mecanizando de forma segura los flujos de registro, hospedaje y facturación.

--- 6.3 DOCUMENTACIÓN TÉCNICA ---

Tecnologías utilizadas:

- Versión de .NET y librerías principales (NuGet)

Microsoft.EntityFrameworkCore.SqlServer

Microsoft.EntityFrameworkCore.Design

Microsoft.EntityFrameworkCore.Tools

Scalar.AspNetCore

- Versión de React y librerías principales (npm)

React 19.2.6

React Router DOM 7.15.1

Axios

- Versión de SQL Server

2022

Arquitectura:

- Descripción de la estructura del proyecto

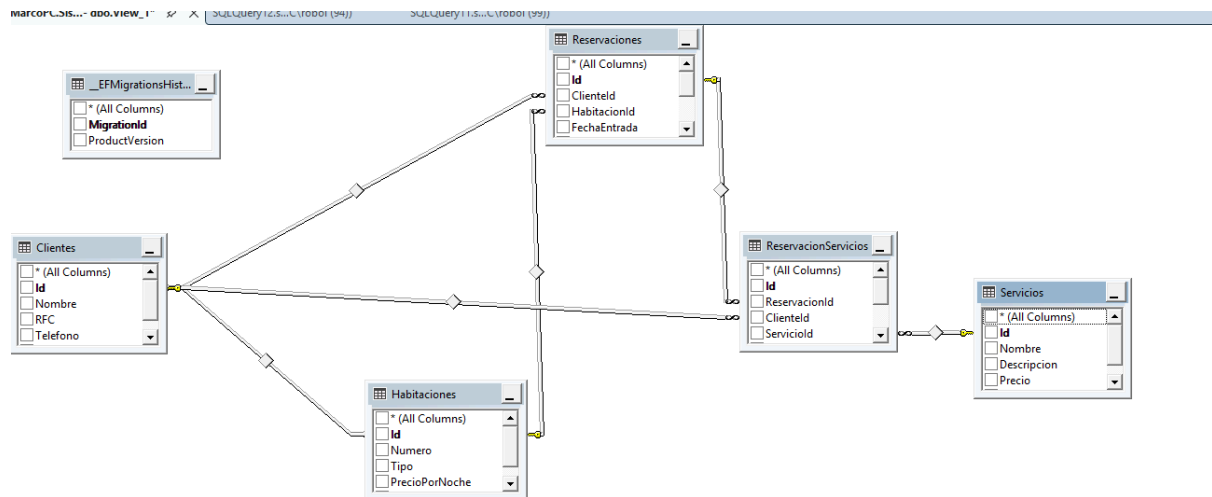
Models ,Controllers, Data, Dto, Migrations, Vistas

- Explicación de la comunicación entre front end y back end

El front se comunica con el backend mediante peticiones HTTP usando **Axios** y la API **fetch** nativa en algunos casos por un compañero. Las peticiones se realizan a los endpoints REST del backend en <http://localhost:5224/api>. Durante el desarrollo se configura un proxy en Vite para redirigir las llamadas de api al servidor .NET, evitando problemas de CORS anteriormente visto. El backend responde en formato JSON y habilita CORS explícitamente para permitir peticiones desde el origen del frontend en <http://localhost:5173>.

Base de datos:

- Descripción de cada entidad y sus atributos principales
- Relaciones entre tablas



API:

- Listado de endpoints desarrollados

Método	Ruta	Descripción
GET	/api/habitaciones	Obtener todas las habitaciones
GET	/api/habitaciones/{id}	Obtener habitación por ID
GET	/api/habitaciones/libres	Obtener solo habitaciones libres
POST	/api/habitaciones	Crear nueva habitación
PUT	/api/habitaciones/{id}	Actualizar habitación
PATCH	/api/habitaciones/{id}/estado	Cambiar estado de habitación
DELETE	/api/habitaciones/{id}	Eliminar habitación

Cientes

Método	Ruta	Descripción
GET	/api/clientes	Obtener todos los clientes
GET	/api/clientes/{id}	Obtener cliente por ID
POST	/api/clientes	Crear nuevo cliente
PUT	/api/clientes/{id}	Actualizar cliente
DELETE	/api/clientes/{id}	Eliminar cliente

Servicios

Método	Ruta	Descripción
GET	/api/servicios	Obtener todos los servicios
GET	/api/servicios/{id}	Obtener servicio por ID
POST	/api/servicios	Crear nuevo servicio
PUT	/api/servicios/{id}	Actualizar servicio

DELETE	/api/servicios/{id}	Eliminar servicio
--------	---------------------	-------------------

Reservaciones

Método	Ruta	Descripción
GET	/api/reservaciones	Obtener todas las reservaciones
GET	/api/reservaciones/{id}	Obtener reservación por ID
GET	/api/reservaciones/cliente/{clienteId}	Reservaciones de un cliente
POST	/api/reservaciones	Crear reservación (valida disponibilidad)
PUT	/api/reservaciones/{id}	Actualizar reservación
PATCH	/api/reservaciones/{id}/estado	Cambiar estado
DELETE	/api/reservaciones/{id}	Eliminar reservación

Consumos de Servicios

Método	Ruta	Descripción
GET	/api/reservacionsservicios	Obtener todos los consumos

GET	/api/reservacionservicios/{id}	Obtener consumo por ID
GET	/api/reservacionservicios/reservacion/{id}	Consumos de una reservación
GET	/api/reservacionservicios/cliente/{id}	Consumos de un cliente
POST	/api/reservacionservicios	Registrar consumo de servicio
PUT	/api/reservacionservicios/{id}	Actualizar consumo
DELETE	/api/reservacionservicios/{id}	Eliminar consumo

--- 6.4 DISTRIBUCIÓN DEL TRABAJO ---

- Tabla con las tareas realizadas por cada integrante

Integrante	Rol Principal	Tareas Clave realizadas
Emmanuel	Catálogos Base y Tareas Directas	Desarrollo CRUD de Clientes y Servicios en Back-End , vistas de gestión correspondientes en Front-End , conexión HTTP API y documentación de endpoints.
Martin	Estructura de Datos, Configuración y Navegación	Configuración inicial de BD, ApplicationDbContext, Data Seeding y migraciones , CRUD de Habitaciones , inicialización de React, Navbar global, componentes reutilizables y

		manejo de errores. Documentación de arquitectura.
Marco	Lógica Compleja y Módulos Funcionales	Modelos core de Reservación y Consumos , controladores con validaciones lógicas complejas de fechas y costos , vistas de Dashboard, módulo de reservaciones interactivas, Check-Out y facturación con IVA. Documentación de relaciones de BD.

Qué desarrolló cada quien en el Back-End (.NET)

- **Emmanuel:**

- Creó el modelo y los Objetos de Transferencia de Datos (DTOs) para la entidad Cliente (Nombre, RFC, teléfono, email).
- Creó el modelo y los DTOs para la entidad Servicio (Spa, restaurante, tours, etc.).
- Desarrolló por completo el controlador ClientesController.cs con sus operaciones CRUD (GET, POST, PUT, DELETE).
- Desarrolló por completo el controlador ServiciosController.cs con sus operaciones CRUD (GET, POST, PUT, DELETE).

- **Martín:**

- Configuro la cadena de conexión en el archivo appsettings.Development.json e inicializó el archivo ApplicationDbContext.cs.
- Creó el modelo Habitacion.cs detallando los atributos de número, tipo, precio por noche y estado.
- Implementó el mecanismo de Data Seeding en EF Core para precargar automáticamente habitaciones muestra, clientes y servicios iniciales de prueba con las migraciones.
- Generó la migración inicial de Entity Framework Core y aplicó los cambios directamente en el servidor SQL Server (dotnet ef database update).
- Habilitó y validó las herramientas de documentación interactiva Swagger/Scalar.
- Desarrolló el controlador HabitacionesController.cs con su CRUD completo para altas, bajas y modificaciones de cuartos.

- **Marco:**

- Creó los modelos de datos para las entidades core del negocio: Reservacion.cs y ConsumoServicio.cs.
- Desarrolló el controlador del Módulo Funcional 1 (ReservacionesController.cs), programando la lógica para agendar estancias y validando del lado del servidor que la habitación no se empalme con otra reservación en el rango de fechas seleccionado.
- Desarrolló el controlador del Módulo Funcional 2 (ConsumosController.cs), estructurando la lógica para cargar servicios adicionales del catálogo a una habitación o reservación activa y recalcular el costo total acumulado.

Qué desarrolló cada quien en el Front-End (React)

● Emmanuel:

- Diseñó e implementó la vista de Gestión de Clientes, incluyendo la tabla de visualización y el formulario adaptado para agregar o editar huéspedes.
- Diseñó e implementó la vista de Gestión de Servicios Adicionales con su respectiva tabla y formulario de administración del catálogo.
- Conectó ambas pantallas operativas mediante llamadas HTTP (fetch/axios) dirigidas a los endpoints correspondientes de la API.

● Martín:

- Inicializó el proyecto de React utilizando Vite dentro del directorio /vista.
- Configuró la navegación global del sistema con react-router-dom y estructuró el menú o Navbar común de la aplicación.
- Desarrolló componentes visuales reutilizables para todo el equipo, tales como Tarjetas para estatus, botones estandarizados y Spinners de carga.
- Diseñó e implementó la vista de Gestión de Habitaciones, entregando un CRUD visual completamente conectado a la API.

● Marco:

- Diseñó e implementó la vista del Dashboard / Inicio, configurando la pantalla principal con indicadores rápidos como el estado de habitaciones ocupadas vs. disponibles en tiempo real.
- Diseñó e implementó la vista del Módulo de Reservaciones mediante un formulario interactivo para la selección de cliente, habitación disponible y rango de fechas.
- Diseñó e implementó la vista de Check-Out y Facturación, permitiendo la búsqueda de reservaciones, el desglose de consumos extras acumulados, el cálculo automático del total con IVA y la simulación del cierre de cuenta del huésped.
-

--- 6.5 RETROALIMENTACIÓN DEL PROYECTO ---

Cada integrante responde individualmente:

- Qué fue lo más difícil de desarrollar y cómo lo resolvieron
 - Qué herramientas o recursos externos consultaron (documentos, videos, etc.)
- Que cambiarían o mejorarían del proyecto si tuvieran más tiempo
- Que aprendieron que no sabían antes de iniciar el proyecto

CASTRO ESPINOZA

- Qué fue lo más difícil de desarrollar y cómo lo resolvieron

Sincronización del código y control de versiones: Al ser tres desarrolladores trabajando simultáneamente en diferentes módulos (catálogos, estructura base y flujos transaccionales), mantener la sincronización con la rama main sin sobrescribir el trabajo de los demás generó fricciones iniciales. Tuvimos algunos conflictos al intentar conectar ramas locales.

- *Solución:* Establecimos una comunicación más estricta antes de cada *merge* y nos apoyamos fuertemente en comandos como *git stash* para resguardar cambios locales temporalmente mientras actualizábamos nuestros repositorios, resolviendo los conflictos de integración de manera segura.

- Que cambiarían o mejorarían del proyecto si tuvieran más tiempo

Diseño UI/UX más pulido: Migraríamos los estilos básicos a una librería de componentes robusta (como Tailwind CSS) para darle un aspecto más moderno y profesional, además de mejorar la adaptabilidad (responsividad) en dispositivos móviles.

- Que aprendieron que no sabían antes de iniciar el proyecto

Manejo avanzado del flujo de trabajo en equipo: Aprendimos a aislar responsabilidades, entendiendo cómo el desarrollo de catálogos base alimenta directamente a los módulos más complejos (reservaciones y consumos), y la importancia de estandarizar la forma en que nombramos y subimos nuestras ramas en el control de versiones.

Aburto Gonzalez Martin Zahid

Qué fue lo más difícil de desarrollar y cómo lo resolvieron

No es que hay algo en concreto que haya sido difícil de desarrollar la principal problemática era que aunque nos dividimos en tareas era difícil pues o no sabían ni lo mínimo de como funcionaba asp y teníamos que corregir lo suyo aparte de que hacer nuestro trabajo era como un tipo de carga doble pues no solo era concentrarme en mi tarea si no que revisar el otro commit que era para ver que tuviera coherencia con lo que andabamos haciendo, aparte que a veces no haber ido a clases se perdía contexto de que hacíamos y algo que debería estar implícito antes de iniciar el proyecto tiene que volverse a explicar haciendo que se pierda tiempo valioso en este proyecto porque hubo errores,

Que cambiarían o mejorarían del proyecto si tuvieran más tiempo

Haria un apartado para cliente una interfaz mas moderna y que no parezca hecha por windows forms aparte de que añadir imágenes algo que parezca que en verdad quieres comprar ahí y si tuviera mas libertad hacer la bd en algún programa como supabase para desplegarse en un entorno real

- Que aprendieron que no sabían antes de iniciar el proyecto

Lo Difícil que puede ser trabajar en equipo

Aguilar Moreno Marco David

- Qué fue lo más difícil de desarrollar y cómo lo resolvieron

Ajustar nuestros tiempo para poder trabajar en el proyecto con los de otras materias y el trabajo al igual que ponernos de acuerdo

Solución: lo solucionamos con un road map para tener una visión individual de los objetivos a corto plazo también github ayudó mucho.

- Que cambiarían o mejorarían del proyecto si tuvieran más tiempo

El planteamiento inicial del proyecto y la definición de tareas

- Que aprendieron que no sabían antes de iniciar el proyecto

El uso de código primero antes que base de datos en Entity framework.

--- 6.6 RETROALIMENTACIÓN DEL CURSO ---

- **Qué temas del curso les fueron más útiles para el proyecto**

Arquitectura de software

Uso de react

Axios

Asp

Entity framework

- **Que temas sintieron que necesitaban mas practica o explicación**

Web webhooks

- **Qué le agregarían o quitarían al curso**

nada

- **Como calificarían el curso del 1 al 10 y por que**

10 , cumple todo lo necesario para crear y entender el desarrollo web

- **Qué temas del curso les fueron más útiles para el proyecto**

La implementación de react-router-dom para la navegación global y el uso de peticiones HTTP (fetch/axios) resultaron indispensables para dar vida a la interfaz e interconectar las vistas con el servidor.

- **Que temas sintieron que necesitaban mas practica o explicación**

Al ser tres integrantes desarrollando componentes de forma simultánea, nos enfrentamos a dificultades técnicas con las ramas que evidenciaron la necesidad de mayor práctica guiada en flujos de trabajo en equipo.

- **Qué le agregarían o quitarían al curso**

Taller de Git para entornos colaborativos: Una sesión práctica enfocada en simular y resolver conflictos de código en equipos de desarrollo.

- **Como calificarían el curso del 1 al 10 y por que**

9, El curso es sumamente robusto y proporciona las herramientas clave para ser competitivos en el desarrollo de software moderno.

- Comentarios adicionales para el profesor

Gracias profe.

6.7 CONCLUSIONES

Conclusión general: El desarrollo de este proyecto web fue un gran desafío técnico que nos permitió consolidar conocimientos esenciales de programación y arquitectura de software, enfrentándonos a la complejidad real de construir una plataforma funcional.

Cumplimiento de objetivos: El objetivo inicial se cumplió con éxito. Aunque enfrentamos una curva de aprendizaje pronunciada al inicio y requerimos una etapa de capacitación intensiva en temas que no dominábamos, logramos superar la incertidumbre y entregar un sistema completamente operativo.

Reflexión sobre el trabajo en equipo: La coordinación fue el mayor reto debido a la falta de tiempo por nuestras respectivas responsabilidades laborales. Sin embargo, logramos superar los problemas de horarios y la nivelación de conocimientos apoyándonos mutuamente, lo que nos dejó una lección clave sobre adaptabilidad, comunicación y compromiso bajo presión.